

Sensory Feedback in Motor Control

Three Models: Equilibrium Point, Optimal Feedback Control, and VITE

Week 3 Lecture — Machine Learning for Neuroscience

Object-Oriented Programming for Computational Motor Control

Lecture Outline

1. Why Sensory Feedback? From Open-Loop to Closed-Loop Control
2. Equilibrium Point Hypothesis: The λ -Model (Feldman, 1986)
3. Optimal Feedback Control: LQG Framework (Todorov & Jordan, 2002)
4. VITE: Vector Integration to Endpoint (Bullock & Grossberg, 1988)
5. Model Comparison: Trajectories, Perturbation Recovery, and Predictions
6. OOP Design: Building a Motor Controller Class Hierarchy
7. Clinical Connections and Preview of Future Weeks

1. Why Sensory Feedback?

In Weeks 1 and 2, our reaching model used **feedforward (open-loop) control**: the entire EMG pattern was pre-programmed before movement onset. This works for well-practiced, unperturbed movements, but fails when the limb is bumped, the load changes, or the target moves. **Closed-loop control** uses sensory information (proprioception, vision) to correct errors online.

Three influential theories propose different mechanisms for how the CNS incorporates feedback:

- **Equilibrium Point (EP):** The CNS specifies a virtual equilibrium position; spring-like reflexes pull the limb toward it
- **Optimal Feedback Control (OFC):** The CNS computes time-varying feedback gains that minimize a cost function
- **VITE:** The CNS continuously integrates a difference vector between target and current position

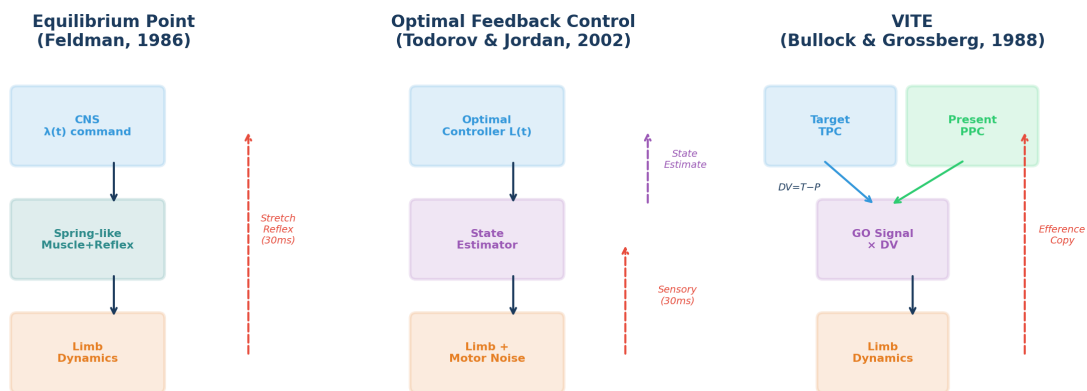


Figure 1: Architecture diagrams for the three feedback models. Red dashed arrows indicate sensory feedback pathways.

2. Equilibrium Point Hypothesis

Feldman (1986) proposed that the CNS controls movement by shifting the **tonic stretch reflex threshold** (λ) of the muscle-reflex system. The muscle-reflex complex acts like a spring: it generates force proportional to the difference between λ and the sensed joint angle. Movement occurs when λ is shifted smoothly (sigmoidally) from the initial to the target position.

Because the stretch reflex operates through the spinal cord, there is a **~30 ms proprioceptive delay** between the actual limb state and the sensory signal that drives the reflex. The reflex also has a velocity-sensitive component (like a dashpot), which provides damping and helps maintain stability despite the feedback delay.

Equation of Motion (simplified single-joint):

$$\tau = K \cdot [\lambda(t) - \theta(t-\Delta)] - B_{\text{reflex}} \cdot \dot{\theta}(t-\Delta) \quad I \cdot \alpha = \tau - B \cdot \dot{\theta}$$

where K is the stiffness of the muscle-reflex system, $\lambda(t)$ is the time-varying threshold (the control signal from the CNS), $\theta(t-\Delta)$ is the delayed sensory feedback ($\Delta \approx 30$ ms), and B_{reflex} provides velocity-dependent damping. The CNS need only specify $\lambda(t)$ — the spring-like reflexes handle the dynamics automatically.

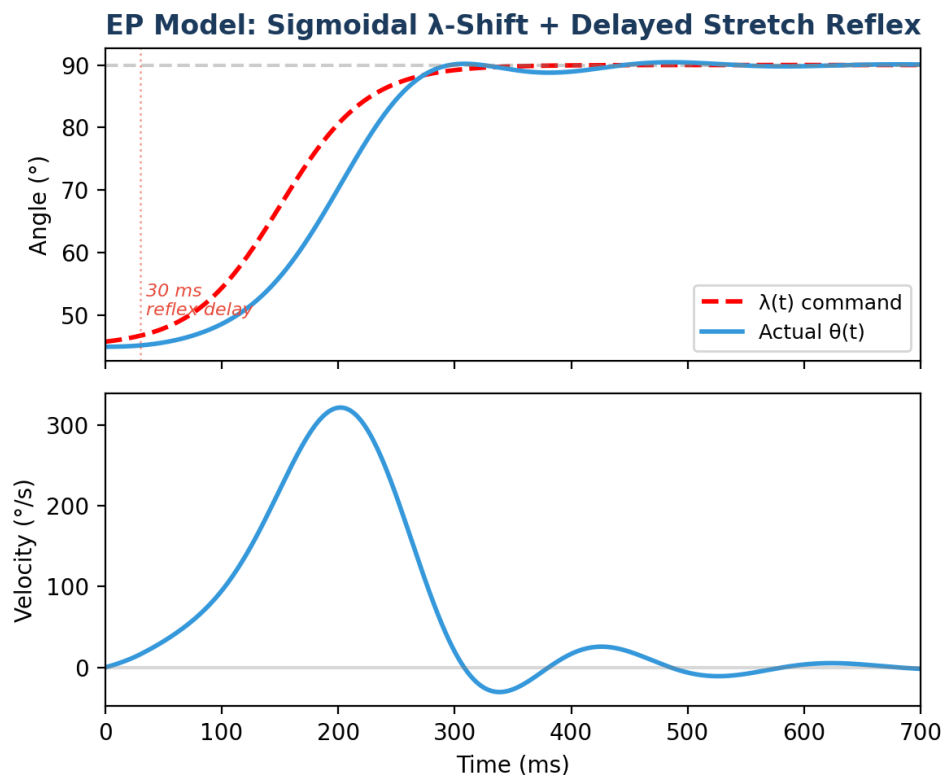


Figure 2: EP model. Red: $\lambda(t)$ ramp command. Blue: resulting joint angle. The limb 'chases' the equilibrium point.

→ **Key Concept: Equifinality** — if the limb is perturbed during movement, the spring-like restoring force automatically brings it back to the equilibrium point. No explicit error computation is needed.

3. Optimal Feedback Control

Todorov and Jordan (2002) proposed that the CNS solves a **stochastic optimal control** problem: given noisy sensors and noisy muscles, find the feedback control law that minimizes a cost function (typically: endpoint accuracy + effort). The full solution combines a **Kalman filter** (state estimator) with **time-varying feedback gains** (from the Riccati equation). For the simple point-to-point movements we simulate here, the deterministic core of OFC already produces realistic bell-shaped velocity profiles.

State dynamics (discrete-time form):

$$x_{t+1} = A \cdot x_t + B \cdot u_t + w_t$$

where $x_t = [\theta, \omega]^T$ is the state vector (joint angle and velocity), A is the state transition matrix encoding plant dynamics (inertia and damping), B maps the control input u_t (torque) to state changes, and w_t is process noise (motor variability).

Optimal control law:

$$u_t = -L_t \cdot x_t \quad (L_t = \text{time-varying feedback gains from Riccati equation})$$

In our simplified implementation, L_t decomposes into a position gain $K_p(t)$ and a velocity gain $K_d(t)$, both of which increase as the movement deadline approaches. A 30 ms sensorimotor delay is included before the first control signal.

Simplified torque equation:

$$\tau(t) = K_p(t) \cdot [\theta^* - \theta(t)] - K_d(t) \cdot \omega(t)$$

where θ^* is the target angle, $\theta(t)$ is the Kalman-estimated position, $\omega(t)$ is the estimated velocity, $K_p(t)$ is the position gain, and $K_d(t)$ is the velocity (damping) gain. Both gains follow a sigmoidal ramp from near-zero to their final values as the movement deadline approaches.

A key prediction is the **minimum intervention principle**: the controller only corrects deviations that affect the task goal. Task-irrelevant variability is left uncorrected, explaining why motor variability is structured rather than random.

→ **Key Concept: OFC predicts that feedback gains increase as the movement deadline approaches — explaining why corrections are stronger late in the movement. For unperturbed point-to-point reaching, the deterministic solution produces clean bell-shaped velocity profiles.**

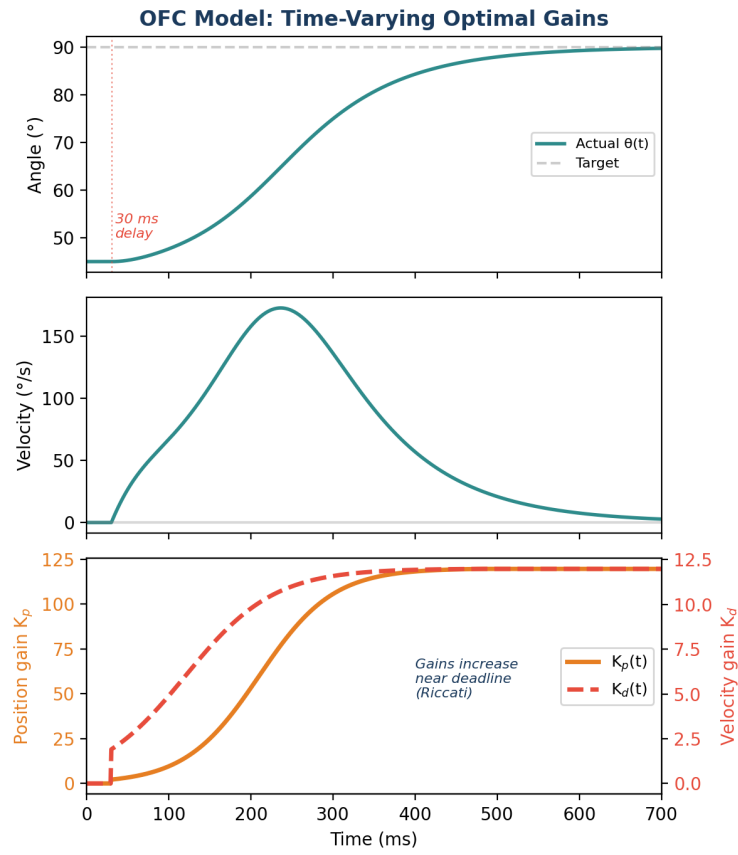


Figure 3: OFC model detail. Top: position trajectory with 30 ms sensorimotor delay at onset. Middle: clean bell-shaped velocity profile. Bottom: feedback gains K_p and K_d increase as the movement deadline approaches (Riccati property).

4. VITE: Vector Integration to Endpoint

Bullock and Grossberg (1988) proposed the VITE circuit with three neural populations: a **Target Position Command (TPC)** representing the desired position, a **Present Position Command (PPC)** that tracks the *planned* position via integration (not the actual limb position), and a **Difference Vector (DV = TPC – PPC)** that drives the movement. A separate **GO signal** gates the integration.

VITE equations:

$$DV(t) = TPC - PPC(t)$$

$$dPPC/dt = \alpha \cdot GO \cdot DV(t)$$

$$\tau = K_{spring} \cdot [PPC(t) - \theta(t)]$$

An important distinction: DV represents the **internal planning error** (TPC – PPC), not the visible gap between target and actual limb angle. PPC converges exponentially toward TPC, so DV decays exponentially. The actual limb θ is driven by the spring between PPC and θ , and lags slightly behind PPC due to plant inertia.

Since DV shrinks as PPC approaches TPC, the rate of change decelerates automatically — producing smooth, bell-shaped velocity profiles. The GO signal allows movements to be planned (DV computed) before execution (GO = 0 during planning, GO > 0 during execution).

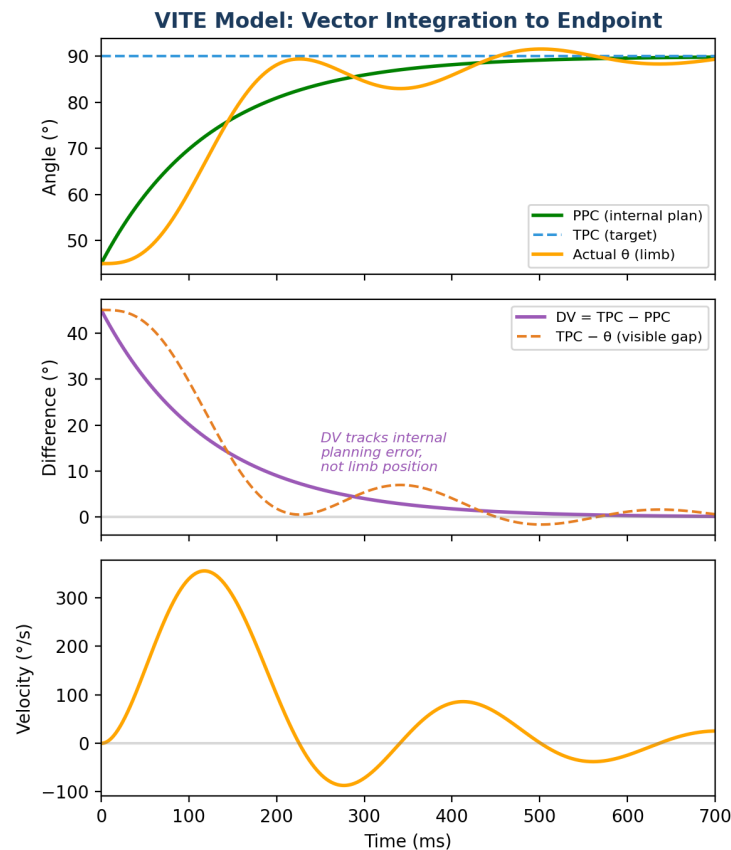


Figure 4: VITE model. Top: PPC (green, internal plan) converges to TPC; actual limb (orange) lags behind. Middle: $DV = TPC - PPC$ decays exponentially (purple); the visible gap $TPC - \theta$ (orange dashed) is larger because θ lags PPC. Bottom: resulting velocity profile.

→ **Key Concept:** The GO signal provides speed control independent of trajectory shape. A larger GO signal produces faster movement with the same bell-shaped velocity profile — explaining speed-accuracy tradeoffs and the ability to freeze mid-movement ($GO \rightarrow 0$).

5. Model Comparison

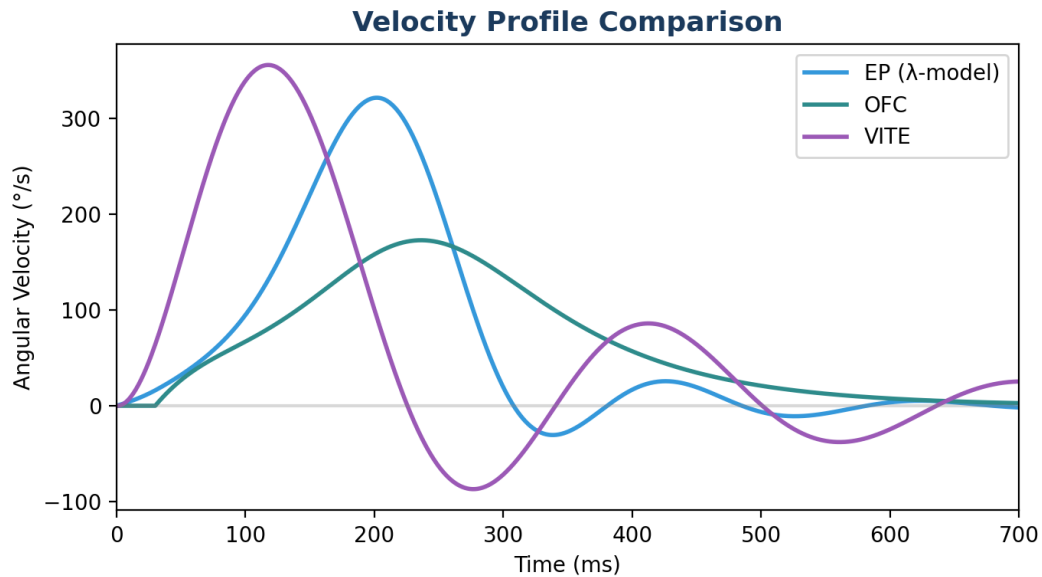


Figure 4: Velocity profiles from all three models. All produce bell-shaped profiles, but with different shapes and peak timings.

Perturbation Recovery

A critical test of any feedback model is how it responds to unexpected perturbations. We simulate a +10° push at 200 ms during movement:

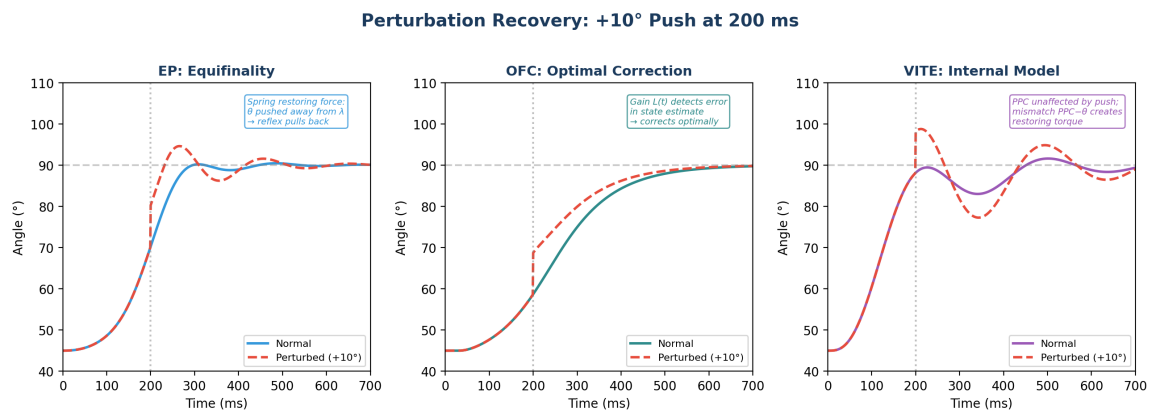


Figure 5: Perturbation response. All three models recover, but through fundamentally different mechanisms.

EP (Equifinality): The perturbation pushes θ away from λ , increasing the stretch reflex error ($\lambda - \theta$). The spring-like restoring force automatically pulls the limb back toward the equilibrium point. No explicit error computation is needed — the correction is an emergent property of the spring-damper system. The 30 ms reflex delay means the correction begins ~30 ms after the push.

OFC (Optimal Correction): The perturbation changes the actual state x_t . The controller immediately computes the new error (target – current state) and applies the time-varying gain L_t to generate a corrective torque. The correction is ‘optimal’ in the sense that it minimizes the cost function (accuracy + effort). The controller does not overcorrect because the velocity gain provides damping.

VITE (Internal Model): The PPC is an internal signal that is *not* directly affected by the physical perturbation. When the limb is pushed (changing θ), the PPC continues its planned trajectory. This creates a mismatch ($PPC - \theta$) that generates a restoring torque via the spring $K_{spring} \cdot (PPC - \theta)$. The recovery mechanism is fundamentally different: it relies on the *discrepancy between internal plan and actual limb position*.

Model Comparison Summary

Feature	EP (Feldman)	OFC (Todorov)	VITE (Bullock)
Control Signal	$\lambda(t)$ threshold	$u(t) = -L_t \cdot \dot{x}$	$GO \times DV$
Feedback	Delayed stretch reflex	State estimate	Efference copy
Delay	~30 ms (spinal)	~30 ms (sensorimotor)	Internal (no delay)
Key Principle	Equifinality	Minimum intervention	Vector integration
Perturbation	Spring restoring force	Optimal correction	$PPC - \theta$ mismatch
Neural Locus	Spinal cord	Cortex + cerebellum	Cortex (M1/PMd)

Figure 6: Summary comparison of the three models across key features.

6. OOP Design for Motor Controllers

All three models share a common structure: they take physical parameters, compute torque from some internal state, simulate the resulting movement, and produce plots. This is a perfect use case for **object-oriented programming** with an **abstract base class** and **inheritance**.

OOP Design: Inheritance Hierarchy for Motor Controllers

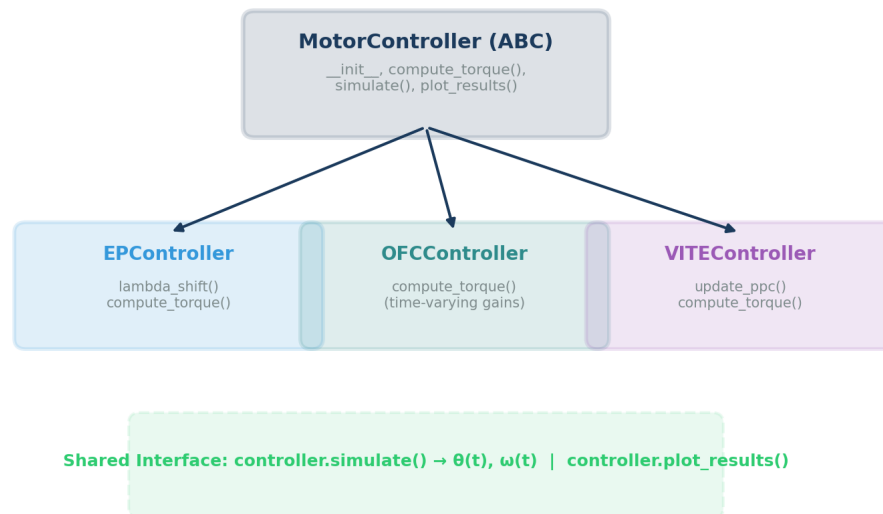


Figure 7: OOP class hierarchy. *MotorController* defines the interface; subclasses implement model-specific logic.

OOP Concepts in This Lab

- **Abstract Base Class (ABC):** *MotorController* defines `compute_torque()` as an abstract method
- **Inheritance:** *EPController*, *OFCController*, *VITEController* inherit from *MotorController*
- **Polymorphism:** All controllers expose the same `.simulate()` and `.plot_results()` interface
- **Encapsulation:** Each model's internal state (λ , Kalman filter, PPC) is hidden inside the class
- **Method Override:** Each subclass overrides `compute_torque()` with its own feedback law

Example usage (polymorphism):

```
models = [EPController(), OFCController(), VITEController()]
for m in models: m.simulate(); m.plot_results()
```

→ **Key Concept:** The power of OOP is that we can add new models (e.g., **Minimum Jerk**, **Internal Model**) by simply creating a new subclass. The simulation and visualization code works automatically because of the shared interface.

VITE: Speed Control via GO Signal

The VITE model uniquely separates **what to move** (the DV) from **how fast to move** (the GO signal). Increasing GO scales the entire velocity profile without changing the trajectory shape or endpoint — a simple mechanism for speed-accuracy tradeoff.

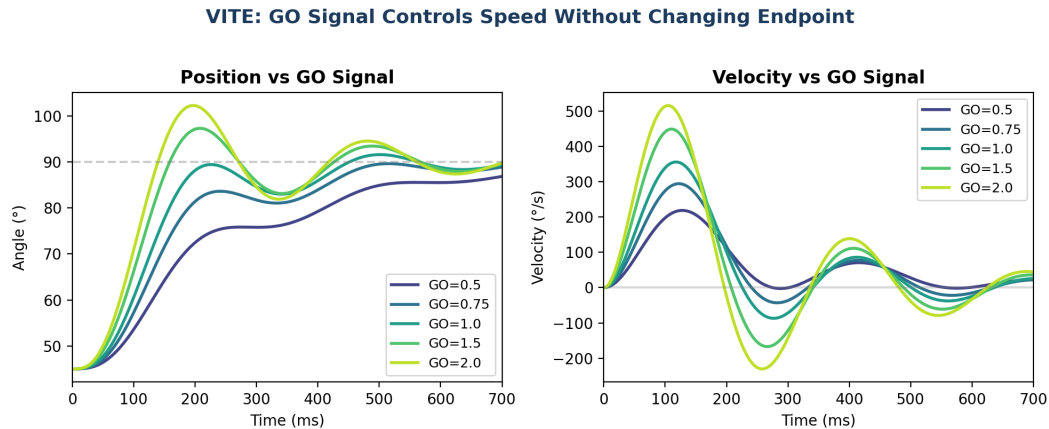


Figure 8: VITE GO signal sweep. Higher GO → faster movement, same endpoint. This explains freezing (GO=0) and speed control in one mechanism.

Extending the Hierarchy: Adding a PD Baseline

The power of the OOP design is demonstrated by adding a simple PD controller as a fourth model — requiring only ~10 lines of code via inheritance:

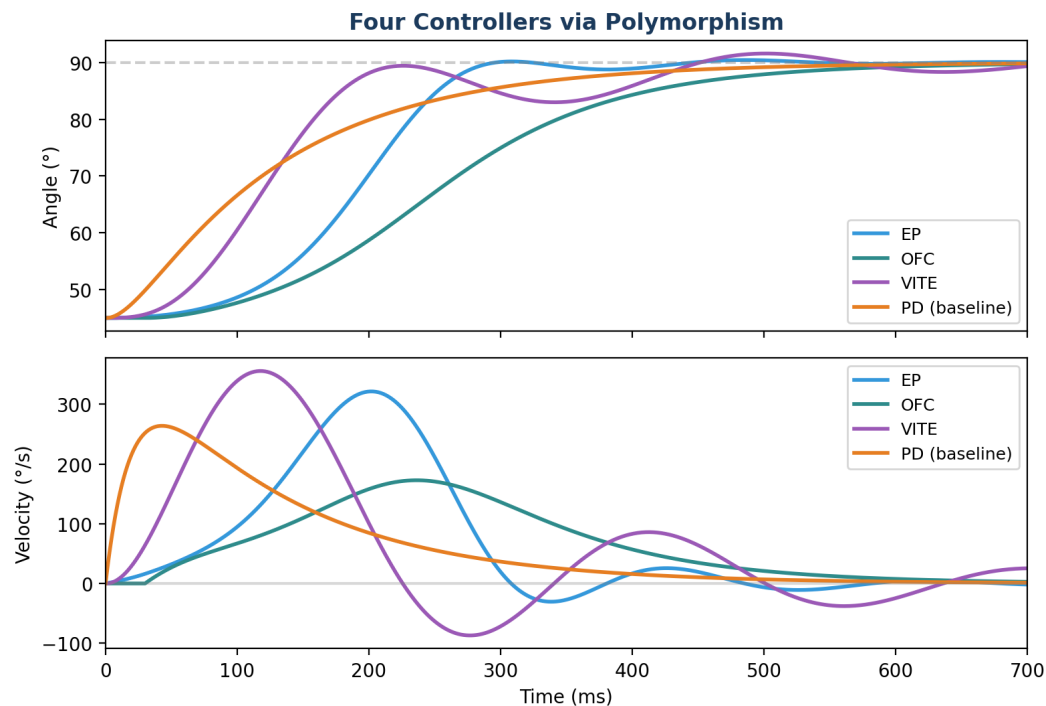


Figure 9: All four controllers (including PD baseline) through the shared polymorphic interface. Note how all reach the target with similar kinematics, but with distinct trajectory shapes.

7. Clinical Connections & Preview

Disorder	EP Interpretation	OFC Interpretation	VITE Interpretation
Spasticity	Abnormal λ threshold	Distorted cost function	Excessive DV gain
Cerebellar ataxia	Impaired stiffness tuning	Noisy state estimation	GO signal variability
Parkinson's	λ shift slowed	High effort cost	Reduced GO signal
Stroke hemiparesis	Spring stiffness reduced	Damaged optimal gains	Weakened DV integration

Each model offers a different lens for interpreting clinical motor disorders. In later weeks, you will use ML classifiers to distinguish between healthy and impaired movement patterns — the OOP class hierarchy you build today will serve as the data generation pipeline.

Key References

- Feldman, A.G. (1986). Once more on the equilibrium-point hypothesis (λ model) for motor control. *J. Motor Behavior*, 18(1), 17–54.
- Todorov, E. & Jordan, M.I. (2002). Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11), 1226–1235.
- Bullock, D. & Grossberg, S. (1988). Neural dynamics of planned arm movements: Emergent invariants and speed-accuracy properties. *Psychological Review*, 95(1), 49–90.
- Harris, C.M. & Wolpert, D.M. (1998). Signal-dependent noise determines motor planning. *Nature*, 394, 780–784.
- Flash, T. & Hogan, N. (1985). The coordination of arm movements. *J. Neuroscience*, 5(7), 1688–1703.